

LA NIÑA: The Network Card Operating System Coherent Interface

Abstract

The fastest packet I/O paths treat the NIC as something to bypass, while the most flexible paths keep the OS in charge and pay higher datapath overhead. Recent work argues that this tradeoff is caused by an artificial split in state: the OS owns scheduling, endpoint, and protection information, while the NIC sees only descriptors, doorbells, DMA addresses, and interrupts. CXL makes that split worth revisiting. If a NIC can participate in host coherence, the interesting object is not only the packet payload, but the small kernel-owned state that decides who owns a buffer, which endpoint should run, and when completion is visible.

This paper presents LA NIÑA, a CXL-oriented cache-coherent NIC architecture that treats the OS/NIC interface as a cache-line and streaming state machine. LA NIÑA uses cache-line ownership to express which actor owns a packet, endpoint, SSD page, or GPU-visible buffer at each instant. Descriptor rings, doorbells, completion queues, and cross-device buffer adapters become compatibility layers above that state machine rather than the architectural contract itself. The current FPGA repository provides a real Ethernet endpoint, a Linux-visible queue contract, and guarded fallback paths; it does not yet claim a completed CXL.cache dataplane. This artifact boundary is intentional. It lets the paper make a precise architectural claim: CXL-CC-NIC is not another zero-copy path, but a lower-level ownership contract that can be measured against MMIO, PCIe-DMA, NV-GDR-style payload placement, and coherent backends without confusing bring-up paths with final performance results.

1 Introduction

The traditional NIC interface is a product of PCIe and distrust. Software constructs descriptors in host memory, notifies the device through a doorbell, the device fetches work and payloads with DMA, and completion state eventually becomes visible to software. This contract has been refined for decades, but it embeds a specific architectural split: the

OS owns scheduling, protection, endpoint, and buffer-lifetime state, while the NIC sees a narrow queue interface that was designed for a non-coherent device.

That split is now the limiting abstraction. Kernel-bypass systems reduce overhead by moving queue polling and packet steering out of the kernel, but they often recover performance by dedicating cores, binding queues to endpoints, and spinning on receive state. Xu and Roscoe argue that the apparent tradeoff between fast bypass and flexible kernel networking is caused by this arbitrary division of state between the OS and the NIC [14]. Their Lauberhorn position pushes the argument to its logical endpoint: with a cache-coherent interconnect and kernel integration, the NIC can use OS state to dispatch RPCs without sacrificing dynamic scheduling.

LA NIÑA takes that thesis and applies it to a CXL-oriented NIC interface. A CXL.cache device can observe and update host memory through coherence rather than through opaque DMA transactions [2]. For a NIC, the most important coherent objects are not only packet payloads. They are the small kernel-owned records that decide packet ownership: descriptors, producer and consumer indices, completion records, endpoint bindings, polling state, and buffer authority. Prior CC-NIC work demonstrated that cache-line-oriented NIC queues can reduce latency and host CPU work [12]. Recent CXL-NIC work goes further by replacing PCIe transactions with CXL.cache accesses and by using CXL.mem-exposed coherent NIC memory for packet buffers and descriptors [6]. The open problem is how to turn these coherent mechanisms into a CXL-era OS/NIC contract with a credible artifact path.

Existing fast I/O mechanisms solve adjacent boundaries. NVIDIA GPUDirect RDMA lets PCIe peers place data directly in GPU memory, removing host-memory staging for payload bytes [9]. Facet/Conduit argues that cross-device pipelines need a route-wide page lifecycle so storage, NICs, GPUs, and CPUs do not rebuild buffer objects at every boundary [1]. Both are useful substrates for LA NIÑA. Neither changes the state split at the NIC queue itself: the NIC is still admitted into the system through descriptors, doorbells, completions, and ownership rules shaped by non-coherent

PCIe.

LA NIÑA argues that the next boundary to redesign is the OS–NIC state interface, and that the right primitive is a cache-line/streaming mode rather than another queue API. A CXL cache-coherent NIC should not merely attach an old PCIe programming model to a coherent link. It should expose queue, endpoint, and cross-device buffer state as coherent cache-line objects, make ownership transfer explicit in memory, and let packets flow as streams of owned cache lines. Descriptor rings, doorbells, completion queues, and subsystem-specific buffer adapters can then be treated as compatibility views over the same state machine. This is the architectural edge of LA NIÑA over NV-GDR-style payload placement and Facet-style page lifecycle management: LA NIÑA targets the lower ownership boundary at which those mechanisms still meet conventional device layers.

The current Agilex7 Type-2 Ethernet repository gives this paper an implementation anchor without requiring it to overclaim. The repository contains a real FPGA Ethernet substrate, a Linux-facing queue abstraction, and defensive capability checks that prevent unavailable backends from being mistaken for measured performance. We therefore use the artifact as a substrate for a CXL-CC-NIC design and evaluation methodology, while clearly separating current validated paths from the future coherent backend.

This paper makes four contributions:

1. It reframes CXL-CC-NIC as an OS/NIC state-sharing problem rather than as a faster payload-copy mechanism.
2. It proposes a coherent control-plane architecture in which queue, completion, endpoint, storage-page, and scheduling records are expressed as a cache-line/streaming ownership state machine.
3. It positions LA NIÑA relative to NV-GDR, Facet/Conduit, CC-NIC, CXL-NIC, Ensō, and Laubhorn, showing that payload placement, route-wide page lifecycle, coherent queues, streaming NIC interfaces, and OS-integrated dispatch solve different boundaries.
4. It grounds the proposal in the current FPGA repository while preserving a conservative line between implemented substrate evidence and projected CXL.cache behavior.

The rest of the paper motivates the state split, describes the architecture and prototype substrate, outlines an HPCA-style evaluation plan, and discusses the limitations that must be resolved before claiming a full CXL cache-coherent NIC.

2 Motivation

2.1 The NIC Boundary Is an OS-State Boundary

The dominant optimization story in high-performance I/O is to avoid moving payload bytes through the CPU. This is necessary, but it is incomplete. A packet arrival is also a scheduling event. The system must decide which endpoint owns the packet, whether the endpoint is running or sleeping, which core should be notified, which buffer lifetime is ending, and which protection domain is allowed to observe the bytes. Today, that state lives mostly in the OS while the NIC sees descriptors and interrupts.

This is why kernel bypass is both successful and unsatisfying. It moves the hot queue path closer to the application, but it often does so by freezing or duplicating OS decisions: endpoints are pre-bound to queues, cores are dedicated, receive paths poll aggressively, and dynamic workload changes require coordination outside the fast path. The cost being avoided is not only packet copying. It is the cost of repeatedly crossing a narrow boundary between NIC queue state and OS scheduling state.

2.2 CXL Makes Fine-Grained State Sharing Plausible

CXL changes the design question. A CXL NIC should not be evaluated only as a faster DMA endpoint. Its architectural opportunity is to make the control path coherent: descriptors, endpoint records, producer and consumer indices, completions, and scheduling hints can be shared as cache-line-owned state. The key question becomes whether the host and NIC can treat these records as typed kernel objects rather than as passive DMA metadata.

This point is already visible in CXL-NIC designs that use CXL.cache for NIC-to-host memory accesses and CXL.mem to expose coherent NIC memory to the host [6]. LA NIÑA builds on that direction, but moves the design target from coherent packet buffers and descriptor placement to the OS-visible state contract around endpoint selection, notification, and ownership.

This does not mean giving the NIC arbitrary access to kernel internals. LA NIÑA uses a narrow state-sharing contract. The kernel publishes explicit endpoint records, queue pages, buffer tokens, and scheduling hints; the NIC consumes and updates only those records according to cache-line ownership rules. The security model is therefore closer to a trusted kernel component with a typed interface than to an untrusted peripheral with broad DMA authority.

2.3 Cache-Line Ownership Can Collapse Device Layers

The stronger claim is that memory itself can express ownership for devices such as NICs and SSDs. Today, each layer has its own admission object: a socket or AF_XDP frame for networking, a `bio` or submission queue entry for block I/O, a GPU peer mapping for NV-GDR, and a completion queue for ownership return. These objects are necessary because the underlying hardware interface does not share a single fine-grained ownership language.

With coherent device access, the common language can instead be the cache line. A packet header, an SSD page token, a GPU-visible response buffer, and an endpoint record can move through a small state machine: produced by one actor, claimed by the next, streamed while data is being consumed, and retired when the completion cache line returns ownership. If the state machine is the interface, the higher layers no longer need to rebuild ownership at every boundary. They become policy and compatibility views over a lower cache-line ownership plane.

This is also why LA NiÑA may be latency-competitive with GPU direct I/O for small messages even when NV-GDR removes a payload copy. NV-GDR optimizes where bytes live; LA NiÑA targets the control transitions that decide when those bytes are ready, which endpoint should run, and when the buffer can be reused. The paper therefore treats “faster than GDR latency-wise” as an evaluation hypothesis: for small or control-heavy transfers, a coherent cache-line state machine may beat a direct-payload path that still pays conventional queue and completion costs.

2.4 Payload Placement Is Not OS Integration

NV-GDR is the right comparison point because it is already a successful direct I/O mechanism for GPU-centric systems. It allows a third-party PCIe device such as a NIC to access GPU memory through peer mappings, avoiding a host-memory staging copy [9]. This is a powerful payload-placement primitive. It does not, by itself, tell the NIC which endpoint is runnable, how completion state should interact with a kernel scheduler, or when a buffer token can be safely reused by a different protection domain.

LA NiÑA targets that missing layer. A future LA NiÑA system may use GPU-visible buffers, just as NV-GDR systems do, but the central optimization is the control edge. The descriptor cache line, the completion cache line, the endpoint record, and the packet metadata cache line become architectural objects. This distinction matters most for small messages, RPC-like services, storage-to-network egress, and GPU-produced responses where the software cost of steering packets can dominate the cost of moving the bytes themselves.

2.5 Facet/Conduit Exposes the Remaining Edge

Facet/Conduit strengthens this argument from a different direction. Its premise is that cross-device pipelines pay not only for copies, but also for repeatedly reconstructing legal buffer objects and ownership lifetimes. By making a registered page the common meeting point for storage, network, GPU, and CPU subsystems, Facet reduces the route-wide cost of moving data across device boundaries [1].

That result is complementary to LA NiÑA because it clarifies what remains below the page lifecycle. A page pool can make the payload object stable across devices, but the NIC still has a queue interface at which work is admitted and completed. If that interface remains PCIe-shaped, the system still pays for a non-coherent NIC edge even when the page object itself is shared. LA NiÑA therefore pushes the Facet/Conduit idea one level lower: the page can be shared across the route, while the NIC queue and endpoint state that name, schedule, and retire the page are also designed for coherent ownership.

2.6 Why the Prototype Must Be Conservative

The architecture is only useful if the artifact can distinguish what is measured from what is projected. A CXL-CC-NIC paper should not hide a missing coherent backend behind software fallback, nor should it present link bring-up traffic as an end-to-end dataplane result. The current repository is valuable because it already encodes that discipline: the Linux-visible queue abstraction exists, the Ethernet substrate is real, and unavailable high-performance paths are guarded rather than silently benchmarked.

This conservative stance is not a weakness of the paper; it is what makes the paper falsifiable. The current prototype can validate the surrounding NIC substrate and software contract. The final CXL contribution will be measured by replacing the PCIe-shaped transport under that contract with coherent queue-and-OS-state ownership and then comparing the same packet interface across MMIO, PCIe-DMA, and CXL-coherent variants.

3 Use Cases

LA NiÑA targets workloads where packet payloads can be placed near the right compute or storage device, but packet control still crosses a conventional NIC boundary and then re-enters the OS for scheduling and ownership decisions.

Small RPC and serverless dispatch For small RPCs, the payload may fit in one or a few cache lines, so software dispatch and wakeup cost can dominate byte movement. A coherent NIC can consume kernel-published endpoint records and decide whether to deliver the request to a polling user thread, notify a kernel worker, or return ownership to a scheduler path.

Table 1: Adjacent mechanisms solve different I/O boundaries.

Mechanism	Boundary optimized	Boundary left to LA NiÑA
NV-GDR	Places payloads directly in GPU memory and avoids host staging.	NIC descriptors, completions, endpoint state, and scheduling remain outside the mechanism.
Facet/Conduit	Maintains a route-wide page lifecycle across storage, network, GPU, and CPU.	The NIC endpoint still admits and retires work through a NIC-specific queue.
LA NiÑA	Makes queue, completion, endpoint, storage-page, and scheduling state coherent cache-line/streaming ownership objects.	Composes with GPU-visible buffers and shared page pools rather than replacing them.

LA NiÑA does not require the NIC to execute the full RPC handler, but it makes the dispatch decision a coherent OS/NIC state transition rather than a descriptor-ring side effect.

NVMe-to-network egress Storage-backed serving pipelines increasingly move data from NVMe into a network stream with limited CPU processing. Existing zero-copy paths reduce payload movement, but queue submission and completion still require explicit software/device coordination. A coherent NIC queue can let the storage pipeline, host software, and NIC exchange buffer ownership with fewer control-path transitions.

GPU-assisted packet generation and validation Inference and analytics services often produce response payloads on a GPU. NV-GDR can help place those payloads, but the NIC still needs a low-cost way to learn which buffers are ready and when they may be recycled. LA NiÑA provides the coherent queue edge that can compose with GPU-visible memory.

Dynamic receive-side pipelines On the receive path, packet data may be consumed by a CPU thread, GPU kernel, or storage writer. Facet-style page lifecycles can keep the payload object stable across those consumers; LA NiÑA adds a coherent NIC queue discipline for the moment at which the NIC produces the packet, selects the next OS-visible endpoint, and releases ownership.

4 Design Principles

LA NiÑA follows seven design principles.

Treat the NIC as a narrow kernel component The NIC should not infer arbitrary OS state and should not receive broad DMA authority over kernel memory. Instead, the kernel publishes explicit endpoint, queue, scheduling, and buffer records that the NIC may read or update. The interface is small enough to validate, but rich enough to avoid rebuilding OS decisions in user-level polling loops.

Make queues coherent objects, not device mailboxes The primary abstraction is the ownership of descriptor, completion, and buffer metadata cache lines. Doorbells and DMA transactions become implementation choices beneath that abstraction, not the defining interface.

Collapse layers into a cache-line state machine The same ownership language should cover NIC packet buffers, SSD-backed pages, GPU-visible response buffers, and OS endpoint records. Subsystem-specific queues remain useful compatibility views, but the architectural handoff is the state transition on the cache line or cache-line stream.

Expose endpoint state as part of the datapath Packet arrival is a scheduling event. LA NiÑA therefore treats endpoint bindings, polling status, runnable state, and notification targets as datapath records, not as hidden policy buried behind interrupts after the packet has already crossed the NIC queue.

Separate payload placement from control coherence LA NiÑA should compose with GPU-visible buffers, storage-backed pages, and shared page pools. These mechanisms decide where payload bytes live. LA NiÑA decides how the NIC and host exchange the authority to transmit, receive, and reuse them.

Design ownership for cache lines Coherence is useful only when producer and consumer fields avoid false sharing and when completion writes do not create hidden read-modify-write traffic. Queue entries should therefore make the handoff unit explicit and align software polling, NIC writes, and buffer lifetime transitions.

Keep the artifact falsifiable The prototype must expose whether a coherent backend is actually present and must report fallback paths separately. This prevents a paper artifact from turning an unavailable hardware path into an accidental performance claim.

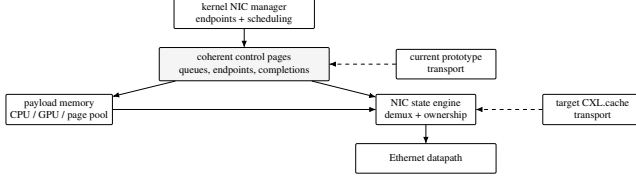


Figure 1: LA NIÑA separates the logical OS/NIC state contract from the transport used to implement it. The current prototype validates the endpoint and software contract; the target CXL backend makes the control pages coherent.

5 Design

5.1 Coherent OS/NIC State Contract

LA NIÑA exposes a set of coherent control pages owned by the kernel and interpreted by the NIC. These pages contain four classes of state. Queue records describe transmit work, receive-buffer availability, and completion slots. Endpoint records map packets to protection domains, queues, and runnable entities. Scheduling records summarize whether an endpoint is polling, sleeping, pinned to a core, or available for kernel wakeup. Buffer records describe payload tokens and the authority to reuse them.

The contract is intentionally typed. The NIC does not chase arbitrary kernel pointers and does not inspect scheduler internals. It consumes compact records with version fields, ownership bits, and capability identifiers. The kernel remains the authority that creates endpoints, changes policy, and revokes NIC access. The NIC is fast because it can observe the state needed for common packet transitions without forcing every transition through an interrupt or a doorbell.

5.2 Coherent Queue Mechanics

LA NIÑA exposes transmit work, receive buffers, and completions as a shared queue region. The host produces descriptors and buffer availability; the NIC consumes them, moves packet ownership, and publishes completion state. The important design choice is that these records are not treated as passive DMA metadata. They are coherence-visible ownership objects whose layout determines the cost of the host–NIC interaction.

This contract preserves the familiar separation between transmit submission, receive buffer provisioning, and completion consumption, but changes the mechanism underneath. A PCIe NIC optimizes by reducing explicit interactions with the device. LA NIÑA optimizes by making each required interaction cheap and local in the coherence protocol.

5.3 Cache-Line/Streaming State Machine

The lowest LA NIÑA abstraction is a state machine over one cache line or a stream of cache lines. Each record carries an owner, a state, a version, and a payload token. **FREE** means the kernel allocator owns the line. **READY** means a producer such as the CPU, NIC, SSD, or GPU-visible runtime has filled the token and release-published ownership. **CLAIMED** means the next actor has acquire-observed the version and may consume the token. **STREAMING** means a multi-line payload is being consumed in order while the head cache line remains the control point. **DONE** returns authority to the kernel or to the next endpoint.

This state machine is deliberately below socket buffers, block descriptors, and GPU peer-memory registrations. A networking stack can view a record as a packet buffer, a storage path can view it as an SSD page token, and a GPU path can view it as a response buffer, but each view is backed by the same ownership transition. The design goal is to avoid re-materializing ownership at every device boundary. The cache line is the rendezvous object; streaming mode is the extension for payloads too large to fit in one line.

5.4 Receive Dispatch Paths

On receive, the NIC first demultiplexes the packet to an endpoint record. If the record says the endpoint is polling on a coherent receive slot, the NIC can publish the packet metadata and buffer token directly into that slot. If the endpoint is runnable but not polling, the NIC publishes a completion and notifies the kernel-visible target. If the endpoint is sleeping or overloaded, the NIC falls back to a kernel queue so the OS can make the scheduling decision.

This design is deliberately less aggressive than full RPC offload. LA NIÑA does not require the NIC to understand application argument formats or jump targets. It focuses on the OS boundary that every packet must cross: endpoint selection, notification, and buffer ownership. More application-specific dispatch can be layered above this contract, but the base claim is that the NIC should share the kernel state needed to avoid unnecessary boundary crossings.

5.5 Ownership and Ordering

The queue layout divides host-produced and NIC-produced state so that each cache line has a clear writer at each phase of its lifetime. The host should not poll fields that the NIC updates through partial writes, and the NIC should not fetch descriptors whose ownership has not been published. Completion records are therefore the main ordering primitive: they return both status and buffer authority to software.

This design also gives a principled place to reason about batching. Batching is still useful for amortizing software overhead, but it is no longer forced by the high cost of every

device notification. A CXL-coherent queue can choose between latency-oriented and throughput-oriented policies without changing the NIC programming model.

5.6 Composing with GPU and Page-Pool Mechanisms

LA NIÑA does not require packet payloads to reside in ordinary host DRAM. Payload buffers may be GPU-visible, storage-backed, or drawn from a registered page pool. The queue contract names these buffers and transfers authority over them; the placement mechanism determines where the bytes live. This is why LA NIÑA is complementary to NV-GDR and Facet/Conduit rather than a replacement for them.

5.7 Prototype Boundary

The current FPGA artifact implements the software-visible endpoint and a real Ethernet substrate, but the coherent transport is still a target backend. The paper therefore treats the prototype as an architectural scaffold: it fixes the queue and endpoint semantics, exposes fallback behavior, and provides a platform on which a CXL.cache-capable backend can be evaluated without changing the higher-level NIC contract.

6 Prototype

The repository prototype is best understood as a substrate for evaluating the LA NIÑA interface, not as the final coherent NIC. Its role is to make the proposed state contract concrete enough to test and hard enough to fake.

6.1 Hardware Substrate

The FPGA image provides a physical Ethernet endpoint and a Type-2 NIC block that can expose queue state to host software. This is sufficient to exercise packet formatting, endpoint reset, link behavior, and the software-visible NIC contract. The implementation also includes descriptor-oriented datapath components that match the intended queue abstraction, but the current image does not yet provide the CXL.cache requester required for the final coherent mode.

6.2 Software State Contract

The Linux side presents the prototype as a conventional network device while preserving the queue semantics that LA NIÑA will use in the coherent backend. This is important for two reasons. First, it lets the artifact test the same logical ownership transitions before the final transport exists. Second, it prevents the coherent design from becoming an isolated hardware block with no operating system integration path.

The intended Linux integration has three visible pieces. A kernel NIC manager allocates queue and endpoint records,

pins or maps the corresponding control pages for the active backend, and revokes records when endpoints are destroyed. A datapath shim converts ordinary transmit, receive, and completion events into the LA NIÑA ownership vocabulary. Capability probes select the active transport: conservative MMIO, descriptor DMA, or the target coherent backend. The same logical endpoint should therefore be runnable across bring-up stages, with the backend recorded in every measurement.

6.3 Guarded Capability Model

The prototype reports whether a high-performance backend is actually present. If it is absent, software uses the conservative path and labels the result as such. This guard is central to the paper’s methodology: it makes the artifact honest by construction and keeps engineering bring-up from being confused with architectural evaluation.

6.4 What the Prototype Does Not Claim

The current repository does not claim that packets are delivered through a completed CXL.cache dataplane, that the NIC observes production Linux scheduler state without mediation, or that coherent dispatch outperforms kernel bypass. Those are the final evaluation targets. The present artifact claim is narrower: the endpoint, queue vocabulary, fallback discipline, and Ethernet substrate are real enough to support a falsifiable CXL-CC-NIC experiment.

7 Evaluation Methodology

The evaluation must answer an architectural question: how much does coherent OS/NIC state sharing change the cost and flexibility of packet I/O once payload placement is held constant? The current repository does not yet answer that question end to end, so this draft separates available artifact evidence from the final experiment suite.

7.1 Current Artifact Evidence

This evidence is valuable because it reduces artifact risk: the paper is not a pure paper design, and the prototype already exposes the boundary that the coherent backend must replace. At the same time, none of these points should be presented as the final LA NIÑA result.

7.2 Final Research Questions

7.3 Final Experiments

The final HPCA submission should include five experiment groups.

Table 2: Current evidence and how it should be used in the paper.

Evidence	What it establishes	What it does not claim
FPGA Ethernet endpoint	The prototype can exercise a real NIC-facing hardware path.	It is not yet a coherent CXL NIC dataplane.
Linux-visible queue contract	Software can drive the intended ownership interface and expose backend capability.	It is not proof that coherent queue state is faster.
Endpoint and fallback discipline	The artifact can label which state path handled a packet or completion.	It does not yet replace production scheduler state with coherent NIC-visible records.
Storage/GPU/network macro paths	The surrounding device pipeline is runnable and measurable.	The numbers are not a substitute for CXL queue experiments.

Table 3: Final experiments should map claims to measurable questions.

RQ	Question
RQ1	What is the latency and CPU-cycle cost of publishing, consuming, and retiring one queue entry across MMIO, PCIe-DMA, and CXL-coherent transports?
RQ2	When endpoint state is visible to the NIC, how often can receive dispatch avoid a kernel wakeup or user-space polling loop under dynamic load?
RQ3	With payload placement held constant, how much benefit comes from the coherent control path in GPU-to-network and storage-to-network pipelines?
RQ4	For small and control-heavy transfers, can a cache-line state machine beat NV-GDR-style direct payload placement on end-to-end latency?
RQ5	What coherence traffic, cache interference, and fairness costs appear as queue depth, endpoint count, and core count scale?

Queue microbenchmarks Measure submission latency, completion visibility, ownership transfer cost, and host CPU cycles for three implementations of the same logical queue: conservative MMIO, PCIe-style DMA, and CXL-coherent queue access.

Endpoint-dispatch experiments Run small-message RPC-like traffic while varying endpoint count, active core count, polling policy, and request burstiness. Compare Linux kernel delivery, user-level bypass or AF_XDP-style polling, and LA NIÑA coherent endpoint records. The important metrics are median and tail dispatch latency, CPU cycles per request, wakeups avoided, and fairness under load changes.

Packet workload sweeps Sweep packet size, queue depth, batching policy, and polling policy. The key result is not only peak bandwidth, but the point at which coherent ownership beats conventional batching for latency-sensitive traffic.

Composed device pipelines Evaluate storage-to-network and GPU-to-network paths with payload placement held con-

stant. This isolates the contribution of LA NIÑA from NV-GDR-style payload placement or Facet-style page lifecycle reuse.

Layer-collapse latency tests Compare a conventional layered path, an NV-GDR payload-placement path, and the LA NIÑA cache-line state-machine path for small responses, SSD-to-network page tokens, and GPU-produced responses. The key metric is not bulk bandwidth but the number of ownership transitions, cache-line round trips, and software notifications on the critical latency path.

7.4 Validity Criteria

A datapoint should be excluded unless the packet stream is correct, device counters agree with software-visible progress, CPU placement and polling policy are recorded, endpoint-state path is recorded, and the active backend is explicitly identified. This is the discipline needed to compare LA NIÑA against Linux zero-copy APIs, AF_XDP, NV-GDR payload placement, and a PCIe-DMA NIC baseline without conflating software fallbacks with coherent hardware behavior.

8 Discussion

Coherence is not sufficient by itself A naive design could make existing PCIe rings cacheable and still inherit most of the old interface cost. LA NIÑA depends on queue layouts, ownership rules, and completion semantics that are intentionally designed for coherence. It also depends on exposing the right OS state; coherent descriptors alone do not remove the dispatch boundary.

Trust must be narrow Saying that the NIC is part of the OS does not mean the NIC can freely read kernel memory. LA NIÑA narrows the trusted interface to kernel-created control pages with explicit endpoint, queue, scheduling, and buffer records. Protection, revocation, and fallback remain kernel responsibilities.

The benefit is workload dependent LA NIÑA should help most when queue-control overhead and ownership latency matter: small packets, low queue depth, many dynamic endpoints, GPU-produced responses, and mixed device pipelines. Large streaming transfers may be dominated by payload bandwidth, in which case NV-GDR or page-pool reuse may capture most of the gain.

Layer collapse is a latency hypothesis The most aggressive version of LA NIÑA removes several software-visible handoff layers and represents the path as a cache-line ownership state machine. This may outperform direct-payload mechanisms for small transfers, but only if coherence round trips, cache interference, and revocation costs stay below the queue and completion overheads they replace. The paper should evaluate this as a latency hypothesis rather than assume it as a property of CXL.

The artifact boundary is intentional The current prototype validates the endpoint and queue contract, but the final claim requires a real coherent backend. The paper should keep this boundary visible so reviewers can separate architectural design, prototype readiness, and measured performance.

Full RPC offload is a higher layer Lauberhorn shows a more aggressive endpoint: the NIC can decode RPCs and return dispatch-ready control data to a waiting core. LA NIÑA instead targets the lower CXL NIC boundary: queue, endpoint, scheduling, and buffer ownership state. A future system could combine the two, but LA NIÑA should not claim application-level RPC semantics until the artifact supports them.

Facet/Conduit is a composition point Facet/Conduit argues for a common page lifecycle across devices. LA NIÑA can use that lifecycle as the payload substrate while changing the NIC endpoint that admits and retires those pages. The two designs therefore address adjacent layers rather than competing abstractions.

9 Related Work

NICs as OS components Xu and Roscoe argue that the performance/flexibility tradeoff in server networking follows from an arbitrary split in state between the OS and the NIC [14]. Their Lauberhorn design uses a cache-coherent interconnect and kernel integration to let the NIC participate in RPC dispatch and scheduling decisions. LA NIÑA adopts the same state-sharing thesis but targets a narrower CXL NIC interface: coherent endpoint, queue, completion, and buffer records rather than full application-level RPC dispatch.

Cache-coherent NIC interfaces CC-NIC is the closest prior work: it argues that coherent interconnects require a host-NIC interface redesigned around cache-line layout, ownership, and signaling rather than a direct reuse of PCIe queue conventions [12]. CXL-NIC then demonstrates a concrete CXL path, using CXL.cache to replace legacy PCIe transactions and CXL.mem to expose coherent NIC memory for packet and descriptor data structures [6]. LA NIÑA differs in focus. It adds explicit OS-facing endpoint and scheduling state to the coherent queue argument and provides an artifact-oriented path for translating that interface into a CXL-era NIC design while preserving a truthful prototype boundary.

CXL and coherent memory systems CXL enables cache-coherent accelerator attachment and memory expansion semantics [2]. Systems such as DirectCXL and Pond study CXL-attached or pooled memory resources [4, 8]. LA NIÑA applies the same broad interconnect shift to packet I/O, where the shared state is latency-sensitive queue metadata and packet buffers rather than only memory capacity.

Fast packet I/O DPDK, netmap, and AF_XDP reduce kernel overhead and expose packet queues more directly to user space [3, 10, 13]. Ensō shows that the packetized NIC/application interface itself can be a mismatch, and replaces fixed-size packet buffers with a streaming interface to reduce software and I/O overheads [11]. These systems still operate above a device interface whose OS-visible scheduling and endpoint state are outside the NIC contract. LA NIÑA treats that OS-NIC interface itself as the object of redesign, while using AF_XDP, Linux zero-copy APIs, and streaming NIC interfaces as important baselines.

GPU direct I/O NVIDIA GPUDirect RDMA enables third-party PCIe devices to access GPU memory directly through peer mappings and NVIDIA driver support [9]. This is valuable for GPU payload movement, but it does not by itself redesign NIC descriptors, completions, doorbells, or cache-line ownership. LA NIÑA is therefore complementary: it can use GPU-visible buffers as a placement target while still changing the coherent NIC control interface.

Cross-device page lifecycles Facet/Conduit proposes a registered host-page pool that is simultaneously presented through native storage, networking, GPU, and CPU descriptors, with ownership advanced through completions [1]. Its argument reinforces LA NIÑA: route-wide zero-copy needs a lifecycle, not merely a faster pairwise copy edge. LA NIÑA attacks the lower NIC-facing boundary that such a lifecycle still meets when the NIC exposes a conventional queue interface.

Near-CPU and programmable NICs NanoPU, Dagger, and other near-data or near-network systems demonstrate the

benefits of tighter CPU/NIC or FPGA integration for latency-sensitive services [5, 7]. LA NIÑA is complementary: it focuses on the OS-visible queue, endpoint, and buffer interface needed for a CXL-attached NIC rather than on a specific RPC or application offload model.

10 Conclusion

LA NIÑA argues that CXL coherence should change the NIC interface, not merely the wire used to reach it. The deeper boundary is the state split between the OS and the NIC: scheduling, endpoint, protection, and buffer-lifetime decisions live in the kernel, while the device sees descriptors and doorbells. NV-GDR improves payload placement and Facet/Conduit improves cross-device page lifecycles, but a conventional NIC queue remains a separate boundary. LA NIÑA makes that boundary the architectural object: queue metadata, endpoint state, completions, scheduling hints, and buffer ownership become coherent cache-line handoffs. The current FPGA repository provides a conservative prototype substrate and a methodology for evaluating this claim without confusing fallback paths with a completed CXL.cache NIC.

References

- [1] Anonymous Author(s). Facet: Making the page the meeting point for cross-device i/o, 2026. Local draft PDF, /home/victoryang00/hetGPU/Conduit (4).pdf.
- [2] Compute Express Link Consortium. CXL 3.0 specification, 2022. <https://www.computeexpresslink.org/>.
- [3] DPDK Project. Data plane development kit. <https://www.dpdk.org/>.
- [4] Donghyun Gouk, Sangwon Lee, Miryeong Kwon, and Myoungsoo Jung. Direct access, high-performance memory disaggregation with DirectCXL. In *2022 USENIX Annual Technical Conference*, pages 287–294, 2022.
- [5] Stephen Ibanez, Alexander Mallery, Serhat Arslan, Thomas Jepsen, Muhammad Shahbaz, Changhoon Kim, and Nick McKeown. The nanopu: A nanosecond network stack for datacenters. In *15th USENIX Symposium on Operating Systems Design and Implementation*, pages 239–256, 2021.
- [6] Houxiang Ji, Yifan Yuan, Yang Zhou, Ipoom Jeong, Ren Wang, Saksham Agarwal, and Nam Sung Kim. Re-architecting end-host networking with CXL: Coherence, memory, and offloading. In *MICRO 2025 - 58th IEEE/ACM International Symposium on Microarchitecture*, Proceedings of the Annual International Symposium on Microarchitecture, MICRO, pages 1809–1823. IEEE Computer Society, 2025.
- [7] Nikolai Lazarev, Shaojie Xiang, Nadav Adit, Zhiru Zhang, and Christina Delimitrou. Dagger: Efficient and fast rpcs in cloud microservices with near-memory reconfigurable nics. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS ’21, pages 36–51, 2021.
- [8] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Shivaram Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. Pond: CXL-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS ’23, pages 574–587, 2023.
- [9] NVIDIA Corporation. GPUDirect RDMA Documentation. <https://docs.nvidia.com/cuda/gpudirect-rdma/>.
- [10] Luigi Rizzo. netmap: A novel framework for fast packet i/o. In *2012 USENIX Annual Technical Conference*, pages 101–112, 2012.
- [11] Hugo Sadok, Nirav Atre, Zhipeng Zhao, Daniel S. Berger, James C. Hoe, Aurojit Panda, Justine Sherry, and Ren Wang. Enso: A streaming interface for NIC-Application communication. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, pages 1005–1025, Boston, MA, July 2023. USENIX Association.
- [12] Henry N. Schuh, Arvind Krishnamurthy, David Culler, Henry M. Levy, Luigi Rizzo, Samira Khan, and Brent E. Stephens. CC-NIC: a cache-coherent interface to the nic. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ASPLOS ’24, pages 52–68, 2024.
- [13] The Linux Kernel Documentation. AF_XDP. https://docs.kernel.org/networking/af_xdp.html.
- [14] Pengcheng Xu and Timothy Roscoe. The NIC should be part of the OS. In *Proceedings of the 20th Workshop on Hot Topics in Operating Systems*, HotOS ’25, 2025. Also available as arXiv:2501.10138.